



riset.ai

FACE RECOGNITION API GUIDELINE

Version

Version	Release Date	Remarks
2.0.0	2024/06/03	Initial Release

Credentials

Header Field	Value	Description
X-Clientid	bima	use this value for X-Clientid header field

Table of Contents

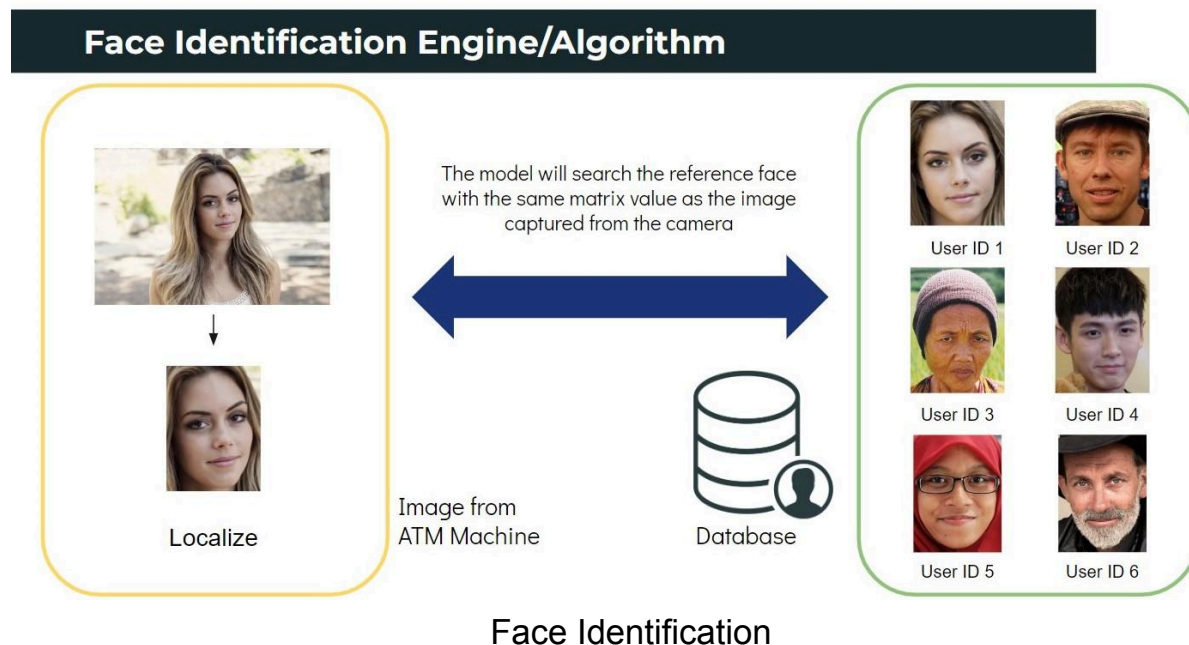
Version.....	2
Table of Contents.....	3
Foundation.....	5
Face Identification.....	5
Face Verification.....	6
Reference Image Requirements.....	7
Device Set-Up Requirements.....	10
Constraints.....	10
Camera Set-Up.....	10
Trigger.....	11
API Documentation.....	12
Base URL.....	12
Postman Collection.....	12
API Endpoints.....	12
GET /facegallery/my-facegalleries.....	12
Request.....	12
Response.....	13
POST /facegallery/create-facegallery.....	14
Request.....	14
Response.....	15
DELETE /facegallery/delete-facegallery.....	16
Request.....	16
Response.....	17
POST /facegallery/enroll-face.....	18
Request.....	18
Response.....	19
GET /facegallery/list-faces.....	20
Request.....	20
Response.....	21
POST /facegallery/verify-face.....	22
Request.....	22
Response.....	23
POST /facegallery/identify-face.....	25
Request.....	25

Response.....	26
DELETE /facegallery/delete-face.....	28
Request.....	28
Response.....	29
POST /compare-images.....	30
Request.....	30
Response.....	31
List of Status Code.....	32

Foundation

Face Identification

This is 1:N authentication means that the system will recognize a person without knowing their identity in the database. This method is superior in user-friendliness because it doesn't need other authentication.



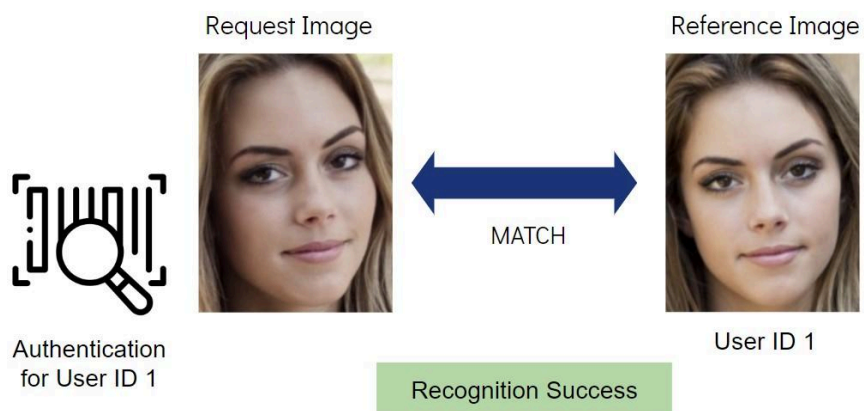
The user's face image is registered into a database (FaceGalleries), contains the user's data from the same group/organization/etc. The system will be able to recognize the user faces after the registration process is completed.

Face Verification

This is 1:1 authentication means that the system ensures the person is actually who claims the identity. This method is more accurate and secure than Face Identification but inferior in user-friendliness due to needing for other authentication like RFID, NFC, KTP, or printed QRCode.



Face Verification Engine/Algorithm



User register by taking a frontal picture of their face and adding a unique identifier such as citizen ID or employee ID to the system. After the registration is completed, the system can verify whether the ID belongs to the correct user.

Reference Image Requirements

To provide more accuracy and precision of facial comparison results, we recommend that you use the following guidelines when choosing **reference photos for registration**.

- Use an image of a face with a straight-on camera.
- Use an image of a face with both eyes open and visible.
- The image should contain the full head and shoulders of the person.
- Avoid items that block the face, such as masks, headbands.
- Use color images.
- Use images of faces with neutral facial expressions with mouth closed and little to no smile.
- Use images with flat lighting on the face.
- Use images that are bright and sharp.
- Use images that have sufficient contrast with the background. A high-contrast monochrome background works well.
- The minimum face resolution is 150 x 150 pixels. Higher-resolution images require a larger minimum face size.
- Format image: JPG (JPEG) and PNG.

Recommended Reference Image



- Face straight to camera
- No occlusion
- Little facial expression
- Good lightning
- Contain one face

Recommended Reference Image

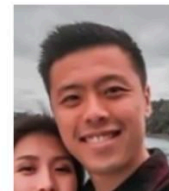
Unrecommended Reference Image



Face is occluded



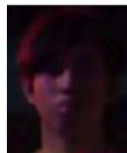
Face is not straight



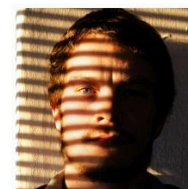
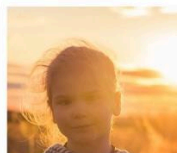
Contain multiple face



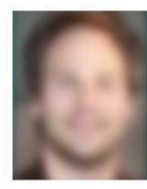
Too expressive



Too Dark or Backlight



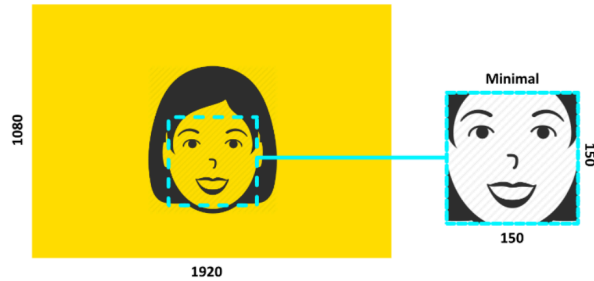
Undistributed light



Blur

Unrecommended Reference Image

Image Recommendations | Resolution



- Image size for face should be above 150x150 pixels for optimal process.
- Any smaller than this, the system will have a hard time detecting and localizing the person's face from the capture image.
- The closer the face is to the camera, the larger the size of captured face, the higher probability of detecting and recognizing face accurately.

Image Recommendations

Device Set-Up Requirements

Constraints

There are several constraints set in order to obtain an optimal performance:

- The camera is placed in a hallway with a minimum length of 3 meters.
- Users are not allowed to wear face attributes other than eyeglasses or a mask.
- If there is a recognition failure when the user is wearing glasses or a mask, the user is expected to take off the glasses or mask in order to provide more features for recognition.
- Users' faces need to be evenly illuminated.
- Users need to walk at an average walking pace.

Image/Camera Set-Up

- **The face resolution must be at least 125 x 125 pixels.** For example, if you have a 360p camera resolution, the distance camera to people should be less than 45 cm.
- **Image size should be no larger than 500KiB.** This is to ensure the best possible latency for sending the image request through the network. Higher sized images will yield slower latency.
- Point the camera to a good lighting environment, not too dark or bright and with no backlight.
- Camera height should be between 1.5 m - 1.75 m.
- If video camera based, frame rate, the recommended frame rate is 30 fps. (at least 25 fps)
- Encoder Bitrate, the recommended encoder bitrate is 3 Mbps.
- Codec, the codec should be h.264 or h.265.
- Give a marker place approximately less than 3 m from the camera to take pictures.



Marker Example

Trigger

Some ideas that can be used to trigger photo submissions to the API:

- Printed QR Code
- RFID
- NFC
- Ultrasonic Module
- Infrared Module
- Face location and minimal size

API Documentation

Base URL

<https://api.riset.ai/face>

Postman Collection

API Endpoints

Example URL for Facegallery Endpoints:

<https://api.riset.ai/face/facegallery/my-facegalleries>

GET **/facegallery/my-facegalleries**

This API gives the list of facegallery.

Request

Headers

KEY	VALUE	VALUE EXAMPLE
X-Clientid	client_id identification	absensi_live

Body

JavaScript
None

Response

Headers

KEY	VALUE
Content-Type	application/json

Body

JavaScript

```
{
  "status": "200",
  "status_message": "Success - FaceGalleries Listed",
  "facegallery_id": [
    "absensi_live@trial",
    "absensi_live@production"
  ]
}
```

Key	Type	Description
status	string	Describing the condition of API hit, please refer to List of Status Code
status_message	string	The verbose message of API hit status, please refer to List of Status Code
facegallery_id	list	List of facegallery

POST /facegallery/create-facegallery

This API creates the new facegallery.

Request

Headers

KEY	VALUE	VALUE	EXAMPLE
X-Clientid	client_id	identification	absensi_live

Body

```
JavaScript
{
  "facegallery_id": "absensi_live@production",
  "trx_id": "alphanumericstring1234"
}
```

Key	Type	Description
facegallery_id	string	Name of facegallery
trx_id	string	Unique transaction identifier, for transaction logging and debugging purposes

Response

Headers

KEY	VALUE
Content-Type	application/json

Body

JavaScript

```
{
  "status": "200",
  "status_message": "Success",
  "facegallery_id": "absensi_live@production"
}
```

Key	Type	Description
status	string	Describing the condition of API hit, please refer to List of Status Code
status_message	string	The verbose message of API hit status, please refer to List of Status Code
facegallery_id	string	Name of facegallery

DELETE `/facegallery/delete-facegallery`

This API deletes a facegallery.

Request

Headers

KEY	VALUE	VALUE EXAMPLE
X-Clientid	client_id identification	absensi_live

Body

JavaScript

```
{
  "facegallery_id": "absensi_live@production",
  "trx_id": "alphanumericstring1234"
}
```

Key	Type	Description
facegallery_id	string	Name of facegallery
trx_id	string	Unique transaction identifier, for transaction logging and debugging purposes

Response

Headers

KEY	VALUE
Content-Type	application/json

Body

JavaScript

```
{
  "status": "200",
  "status_message": "Success",
  "facegallery_id": "absensi_live@production"
}
```

Key	Type	Description
status	string	Describing the condition of API hit, please refer to List of Status Code
status_message	string	The verbose message of API hit status, please refer to List of Status Code
facegallery_id	string	Name of deleted facegallery

POST /facegallery/enroll-face

This API registers a user to the database.

Request

Headers

KEY	VALUE	VALUE EXAMPLE
X-Clientid	client_id identification	absensi_live

Body

```
JavaScript
{
  "user_id":      "absensi_live1234",
  "user_name":    "Person Username1",
  "facegallery_id": "absensi_live@production",
  "image":        "/9j/4AAQSkZJRgABAQEASABIAAD/4...",
  "trx_id":       "alphanumericstring1234"
}
```

Key	Type	Description
user_id	string	Unique user identifier, alphanumeric (eg. #NIK)
user_name	string	The name of the person who has the user_id
facegallery_id	string	Unique FaceGallery identifier, alphanumeric (eg. LocationName, CompanyName, etc)
image	string	Base64 encoded JPG or PNG image

<code>trx_id</code>	<code>string</code>	Unique transaction identifier, for transaction logging and debugging purposes
---------------------	---------------------	---

Response

Headers

KEY	VALUE
Content-Type	application/json

Body

JavaScript

```
{
  "status": "200",
  "status_message": "Success"
}
```

Key	Type	Description
<code>status</code>	<code>string</code>	Describing the condition of API hit, please refer to List of Status Code
<code>status_message</code>	<code>string</code>	The verbose message of API hit status, please refer to List of Status Code

GET /facegallery/list-faces

This API gives a list of the registered user.

Request

Headers

KEY	VALUE	VALUE EXAMPLE
X-Clientid	client_id identification	absensi_live

Body

JavaScript

```
{
  "facegallery_id": "absensi_live@production",
  "trx_id":         "alphanumericstring1234"
}
```

Key	Type	Description
facegallery_id	string	Unique FaceGallery identifier, alphanumeric (eg. LocationName, CompanyName, etc)
trx_id	string	Unique transaction identifier, for transaction logging and debugging purposes

Response

Headers

KEY	VALUE
Content-Type	application/json

Body

JavaScript

```
{
  "status": "200",
  "status_message": "Success",
  "faces": [
    {
      "user_id": "absensi_live1234",
      "user_name": "Person Username1"
    },
    {
      "user_id": "absensi_live5678",
      "user_name": "Person Username2"
    }
  ]
}
```

Key	Type	Description
status	string	Describing the condition of API hit, please refer to List of Status Code
status_message	string	The verbose message of API hit status, please refer to List of Status Code
faces	list	List of registered user

POST /facegallery/verify-face

This API verifies an `user_id` and an image with a registered user or it does 1:1 authentication.

Request

Headers

KEY	VALUE	VALUE EXAMPLE
X-Clientid	client_id identification	absensi_live

Body

JavaScript

```
{
  "user_id":      "absensi_live1234",
  "facegallery_id": "absensi_live@production",
  "image":        "/9j/4AAQSkZJRgABAQEASABIAAD/4Q...",
  "trx_id":        "alphanumericstring1234"
}
```

Key	Type	Description
<code>user_id</code>	<code>string</code>	Unique user identifier, alphanumeric (eg. #NIK)
<code>facegallery_id</code>	<code>string</code>	Unique FaceGallery identifier, alphanumeric (eg. LocationName, CompanyName, etc)
<code>image</code>	<code>string</code>	Base64 encoded JPG or PNG image
<code>trx_id</code>	<code>string</code>	Unique transaction identifier, for transaction logging and debugging purposes

Response

Headers

KEY	VALUE
Content-Type	application/json

Body

JavaScript

```
{
  "status": "200",
  "status_message": "Success",
  "user_name": "Person Username",
  "similarity": 1,
  "masker": false,
  "verified": true,
}
```

Key	Type	Description
status	string	Describing the condition of API hit, please refer to List of Status Code
status_message	string	The verbose message of API hit status, please refer to List of Status Code
user_name	string	Username
similarity	float	Describe the comparison of facial similarities, scale 0.0 to 1.0 (from 0% to 100% similar)
masker	boolean	If a person's face wearing a mask, will return True, else return False

<code>verified</code>	<code>boolean</code>	If similarity above set config parameter(eg. threshold = 0.75), return True, else return False
-----------------------	----------------------	--

POST /facegallery/identify-face

This API identify an image with a registered user or it do1:N authentication.

Request

Headers

KEY	VALUE	VALUE EXAMPLE
X-Clientid	client_id identification	absensi_live

Body

```
JavaScript
{
  "facegallery_id":  "absensi_live@production",
  "image":           "/9j/4AAQSkZJRgABAQEASABIAAD/...",
  "trx_id":           "alphanumericstring1234"
}
```

Key	Type	Description
facegallery_id	string	Unique FaceGallery identifier, alphanumeric (eg. LocationName, CompanyName, etc)
image	string	Base64 encoded JPG or PNG image
trx_id	string	Unique transaction identifier, for transaction logging and debugging purposes

Response

Headers

KEY	VALUE
Content-Type	application/json

Body

JavaScript

```
{
  "status": "200",
  "status_message": "Success",
  "status_description": "Face Recognition Success",
  "return": [
    {
      "confidence_level": "0.943697",
      "mask": "true",
      "user_id": "absensi_live1234",
      "user_name": "Person Username1"
    }
  ]
}
```

Key	Type	Description
status	string	Describing the condition of API hit, please refer to List of Status Code
status_message	string	The verbose message of API hit status, please refer to List of Status Code

<code>confidence_level</code>	<code>float</code>	Describe confidence of model, scale 0.0 to 1.0 (from 0% to 100% confidence)
<code>mask</code>	<code>boolean</code>	If a person's face wearing a mask, will return True, else return False
<code>user_id</code>	<code>string</code>	Unique user identifier, alphanumeric (eg. #NIK)
<code>user_name</code>	<code>string</code>	The name of the person who has the <code>user_id</code>

DELETE [/facegallery/delete-face](#)

This API deletes a user.

Request

Headers

KEY	VALUE	VALUE	EXAMPLE
X-Clientid	client_id	identification	absensi_live

Body

JavaScript

```
{
  "user_id":      "absensi_live1234",
  "facegallery_id": "absensi_live@production",
  "trx_id":      "alphanumericstring1234"
}
```

Key	Type	Description
<code>user_id</code>	<code>string</code>	Unique user identifier, alphanumeric (eg. #NIK)
<code>facegallery_id</code>	<code>string</code>	Unique FaceGallery identifier, alphanumeric (eg. LocationName, CompanyName, etc)
<code>trx_id</code>	<code>string</code>	Unique transaction identifier, for transaction logging and debugging purposes

Response

Headers

KEY	VALUE
Content-Type	application/json

Body

JavaScript

```
{
  "status": "200",
  "status_message": "Success"
}
```

Key	Type	Description
status	string	Describing the condition of API hit, please refer to List of Status Code
status_message	string	The verbose message of API hit status, please refer to List of Status Code

POST /compare-images

This API compares the two images to determine if they are verified or not. This API does not use the information in the database.

Request

Headers

KEY	VALUE	VALUE EXAMPLE
X-Clientid	client_id identification	absensi_live

Body

```
JavaScript
{
  "source_image": "/9j/4AAQSkZJRgABAQEASABIAAD/4QBMRXhpZgAA...",
  "target_image": "/9j/4AAQSkZJRgABAQEASABIAAD/389sdAxcZisA...",
  "trx_id":       "alphanumericstring1234"
}
```

Key	Type	Description
source_image	string	Base64 encoded JPG or PNG of compared image
target_image	string	Base64 encoded JPG or PNG of the reference image
trx_id	string	Unique transaction identifier, for transaction logging and debugging purposes

Response

Headers

KEY	VALUE
Content-Type	application/json

Body

JavaScript

```
{
  "status": "200",
  "status_message": "Success",
  "similarity": 0.8308,
  "verified": true,
  "masker": true
}
```

Key	Type	Description
status	string	Describing the condition of API hit, please refer to List of Status Code
status_message	string	The verbose message of API hit status, please refer to List Status Code
similarity	float	Describe the comparison of facial similarities, scale 0.0 to 1.0 (from 0% to 100% similar)
verified	boolean	If similarity above set config parameter(eg. threshold = 0.75), return True, else return False
masker	boolean	If a person's face wearing a mask, will return True, else return False

List of Status Code

Status Code	Type	Description
200	Success	Success messages
400	General Error	Request malformed
401	General Error	Not authorized
403	General Error	Requested resource denied
411	Business Process Warning	Face not verified or unregistered
412	Business Process Warning	Face not detected
413	Business Process Warning	Face too small
415	Resource Not Found	user_id not found
416	Resource Not Found	facegallery_id not found
451	Resource Not Found	image is null
452	Data Format Error	user_id is null
453	Data Format Error	user_name is null
454	Data Format Error	facegallery_id is null
455	Data Format Error	target_image is null
456	Data Format Error	source_image is null
490	Image Error	Cannot decode image base64
491	Image Error	image type not recognized
492	Image Error	Cannot decode target_image base64
493	Image Error	image error
494	Image Error	Cannot decode source_image base64
495	Image Error	source_image type not recognized